OOP Concepts in Java

17 April 2026 05:23

Object oriented programming is a paradigm that provides concepts such as inheritance, data binding, polymorphism, etc.

OOPs(Object Oriented Programming)
Object Oriented programming aims to implement a real world entities, for example objects, classes, abstraction, inheritance, polymorphism, etc.

Object means a real world entity such as. Pen, chair, table, computer, watch, etc.
Object oriented programming is a methodology or paradigm for designing a program using classes and objects. It simplifies software development and maintenance by providing some concepts:

- Object
- Class
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation

Apart from these concepts, there are some other terms which are used in object oriented design:

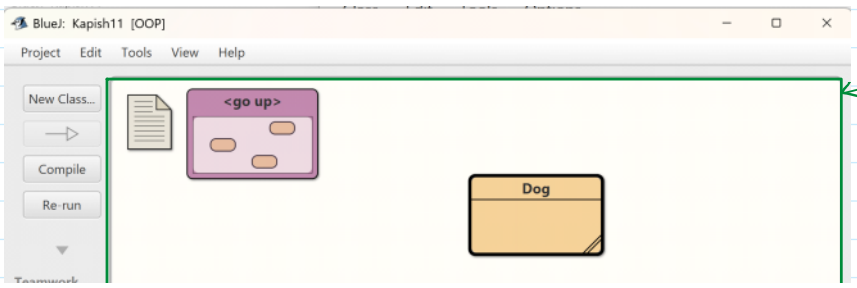
- Coupling
- Cohesion
- Association
- Aggregation
- Composition

What is an Object?
Any entity that has state and behavior is known as an object. For example, a chair, pen, table, keyboard, car, etc. It can be physical or logical.

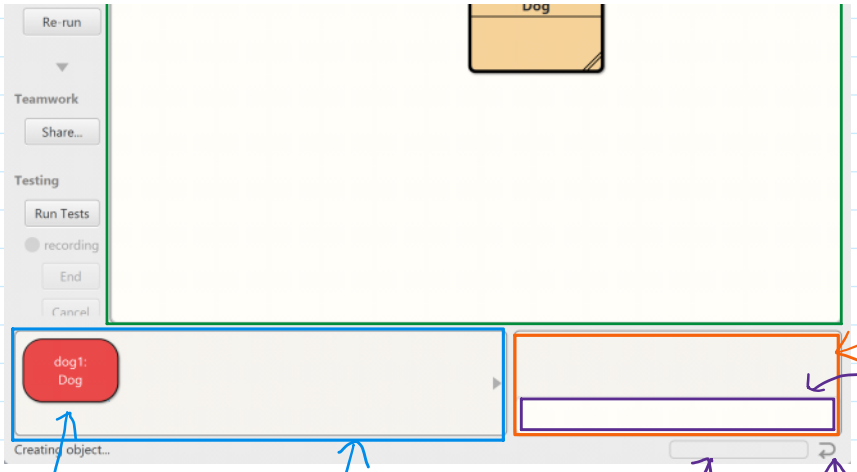
An object can be defined as an instance of a class. It contains an address and takes up some space in memory. Objects can communicate without knowing the details of each other's data or code. The only necessary thing is the type of message accepted and the type of response returned by the object.

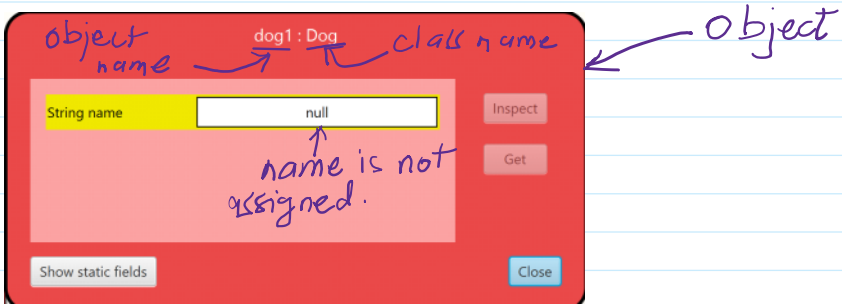
The primary elements of an objects are:

- **State:** It is embodied in an object's properties. Additionally, it reflects an object's attributes.
- **Behavior:** An object's methods are a representation of it. Additionally, it represents how an object reacts to other object.
- **Method:** A method is a group of statements that carry out a certain operation and give an outcome. A method can compete a certain task without giving back any results. Methods are regarded as time saver since they enable us to reuse the code without having to type it again.

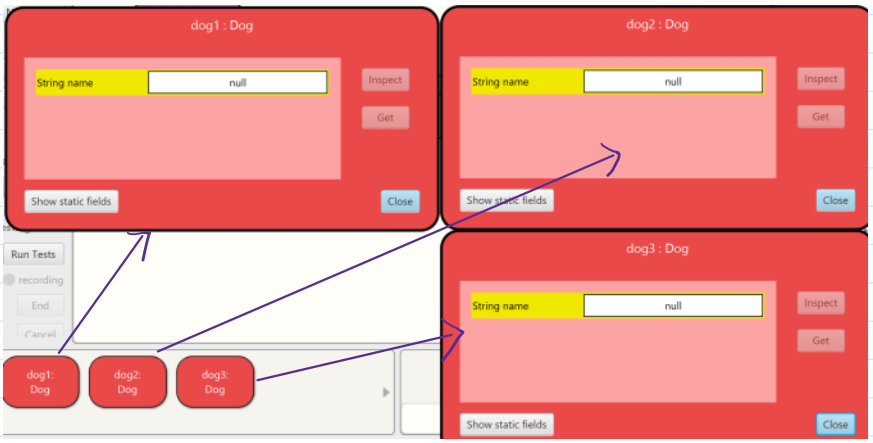


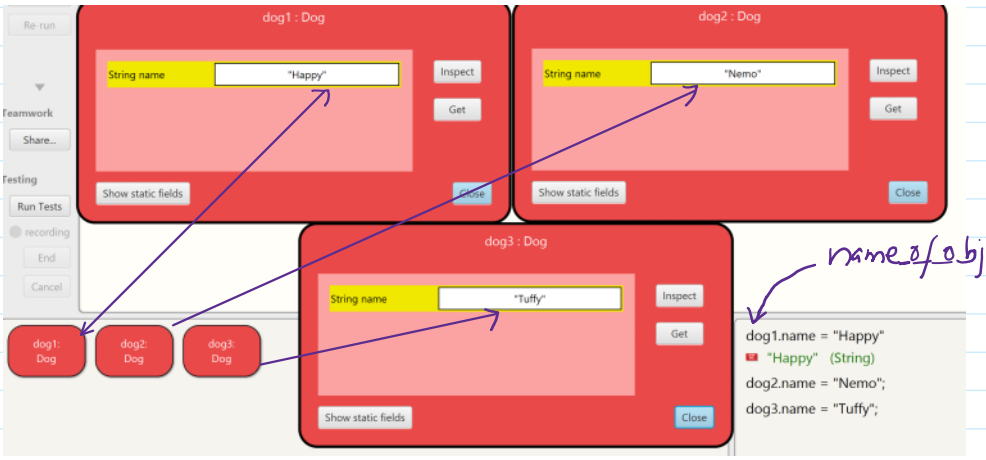
class or package bench



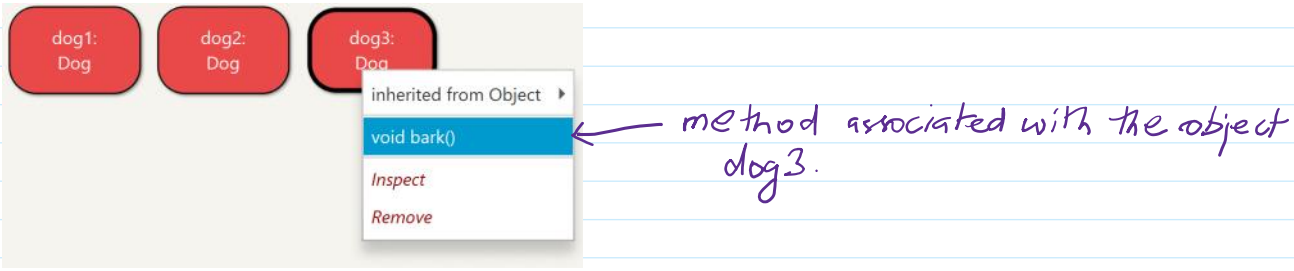


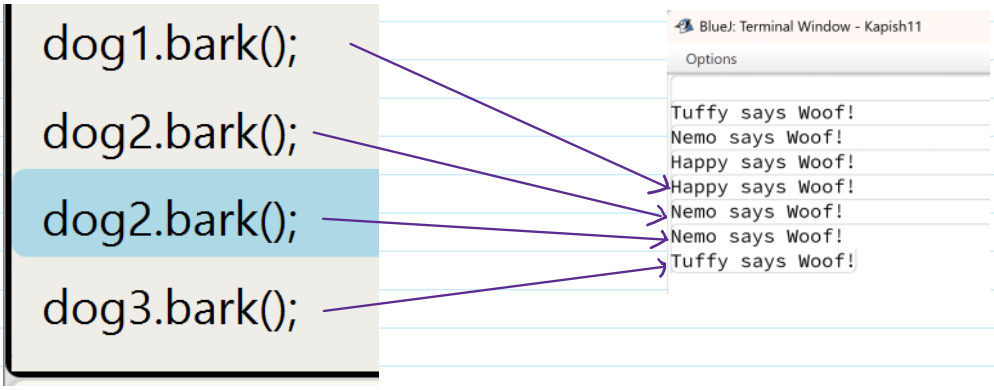
We created three objects of Dog class





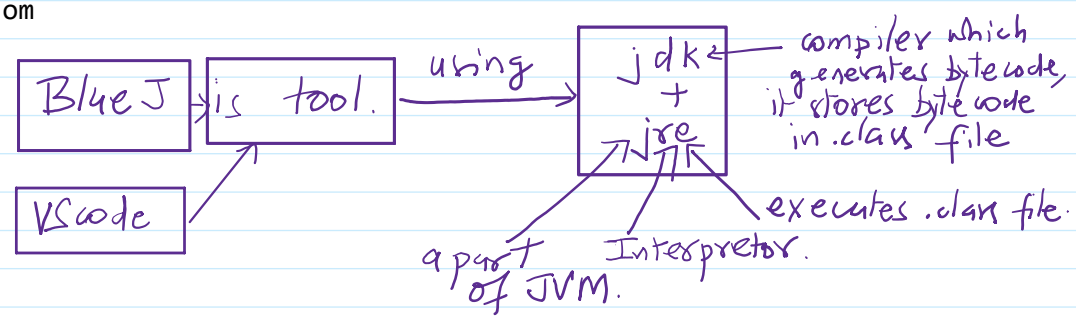
object_name.bark()





What is an IDE?
Integrated Development Environment, it contains code editor or other tools to compile, test and execute the code under one umbrella. For example, Bluej, VSCode, IntelliJ Idea, Eclipse, Netbeans etc.

bluejteacher@gmail.com



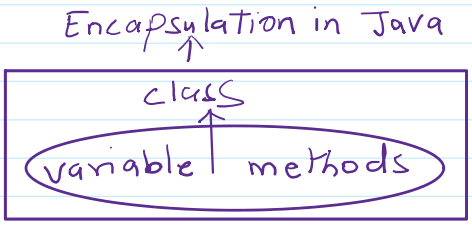
Class

A class is a logical entity. A class can also be defined as a blueprint from which we can create an individual object. Class does not consume any space in the memory.

In general, a class declaration includes the following in the order as it appears:

1. **Modifiers:** A class can be public or have default access.
2. **class keyword:** The class keyword is used to create a class.
3. **Class name:** The name must begin with initial letter (capitalized by convention.)
4. **Body:** The class body surrounds by braces {}.

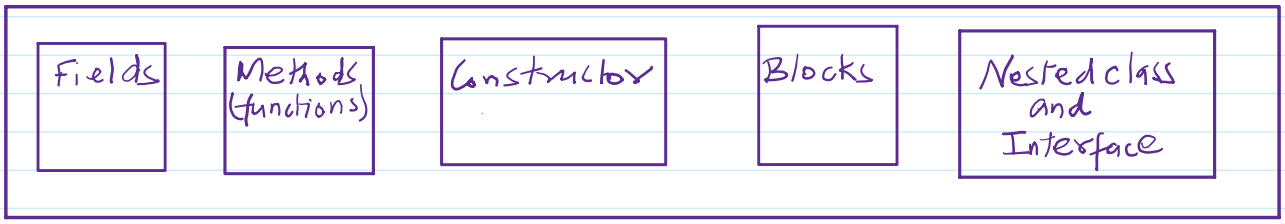
Encapsulation: Binding or wrapping code and data together into a single unit, known as encapsulation. For example, a capsule is wrapped with different medicines.



A Java class is an example of encapsulation.
A Java bean is fully encapsulated class,
because all data members are private.

What is a class in Java?
A class is a group of objects that have common properties. It is a template or blueprint from which objects are created. We can say that it is an object factory. It is a logical entry. It can't be physical.

A Java class contains.

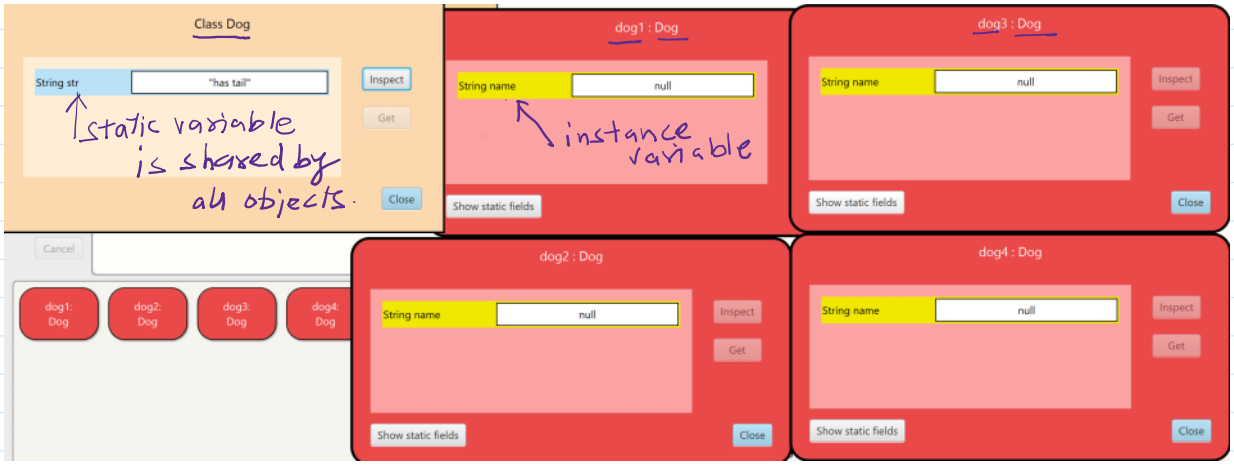


1. **Fields**
   Variables stated inside a class that indicates the status of an object formed from that class are called fields. Sometimes referred to as instance variables. They specify the data that will be stored in each class object. Different access modifiers such as public, private and protected can be applied to fields to regulate their visibility and usability.

2. **Methods**
   Methods or functions defined inside a class that include the action or behavior that objects of that class are capable of performing. These techniques allow the outside world to function and change the object state (fields) Additionally, methods can be void (that is they return nothing) or have different access modifiers. They can also return values.

3. **Constructors**
   Constructors are unique methods that are used to initialize class objects. When an object of the class is created using the new keyword, they are called with the same name as the class. Constructors can initialize the fields of an object or carry out any additional setup, that is required when an object is created.

4. **Blocks**
   Within a class, Java allows two different kinds of blocks: Instance block, commonly referred to as initialization blocks and static blocks. Static blocks, which are usually used for static initialization, are only executed once when the class is loaded into memory. Instance block can be used to initialize instance variable and are executed each time a class object is generated.




```
package OOP;


/**
 * Write a description of class Dog here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Dog
{
    String name; //Field or instance variable
    static String str = "has tail";

    {
        //static block
        System.out.print("\nI am a static block");
        System.out.print("\nNotifying you that object is created!");
    }
    public void bark(){ //method or function
        System.out.print("\n" + name + " says Woof!");
        System.out.print("\n" + str);
    }
}
```

5. Nested Class and Interface

Java permits the nesting of classes and interface inside other classes and interfaces. The members (fields, Methods) of the enclosing class are accessible to nested classes, which can be static or non-static. Nested interfaces can be used to logically group related constants and methods together because they are implicitly static.

Syntax:

```
class <class_name>{
  fields;
  methods();
  //nested class
  class <class_name>{
     fields;
     methods();
  }
```

```
    …
}
```

What is an object in Java?

An object is a real world entity that has state and behavior. In other words, an object is a tangible thing that can be touch and feel like a car or chair, etc. The student management system is an example of an intangible object. Every object has distinct identity, which is usually implemented by a unique ID that the JVM uses internally for identification.

Characteristics of an object

- **State:** It represents data (value) of an object.
- **Behavior:** It represents the behavior (functionality) of an object such as deposit, withdraw etc.
- **Identity:** An object's identity is typically implemented via a unique ID. The ID's value is not visible to the external user. However, it is internally used by the JVM to identify each object uniquely.

Object name

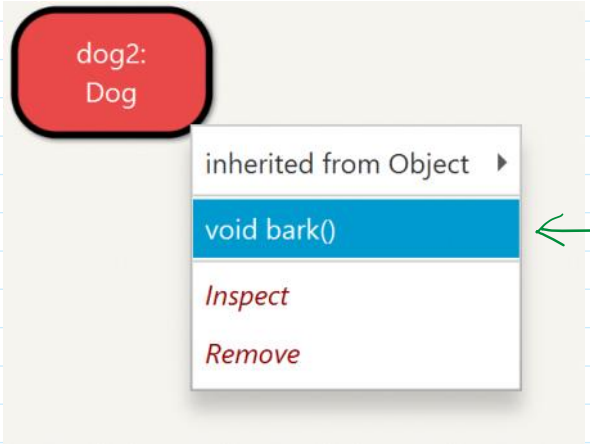
```
System.out.print("\n" + dog1);
```

Output:

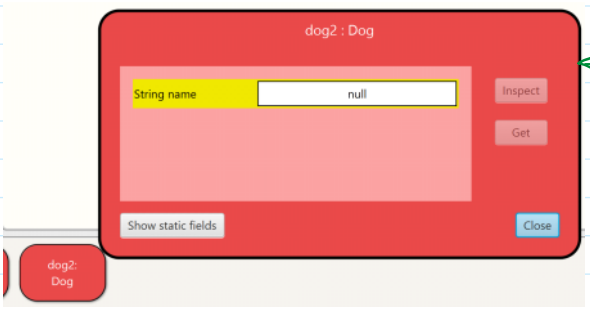
```
OOP.Dog@482178ce
```

Hexadecimal value is Object_id

package Class separator

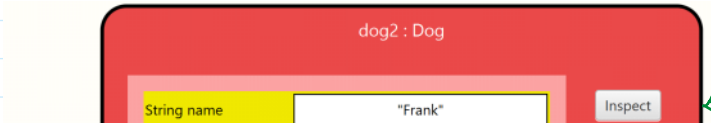


behavior

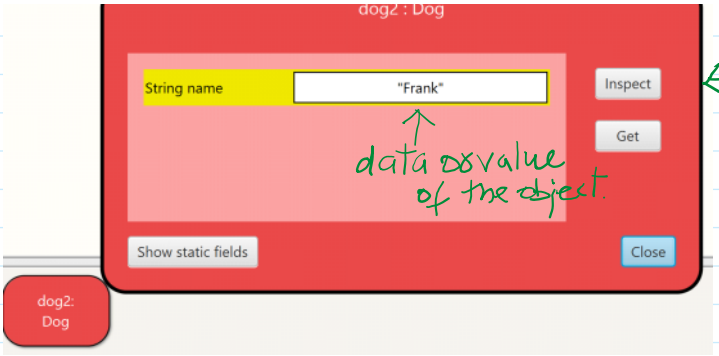


state of the object

```
dog2.name = "Frank";
```



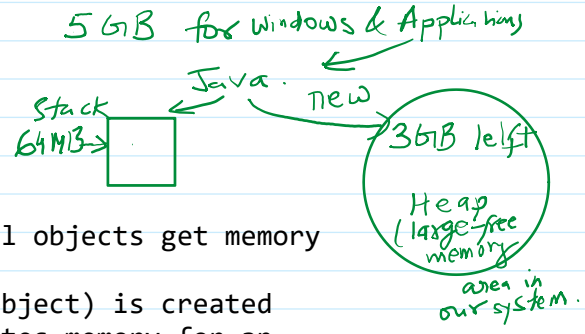
state of this object is changed.



An object is an instance of a class. A class is a template or blueprint from which objects are created. So an object is the instance (result) of a class.

Objects Definitions

- An object is a real world entity.
- An object is a runtime entity.
- The object is an entity that has state and behavior.
- The object is an instance of a class.

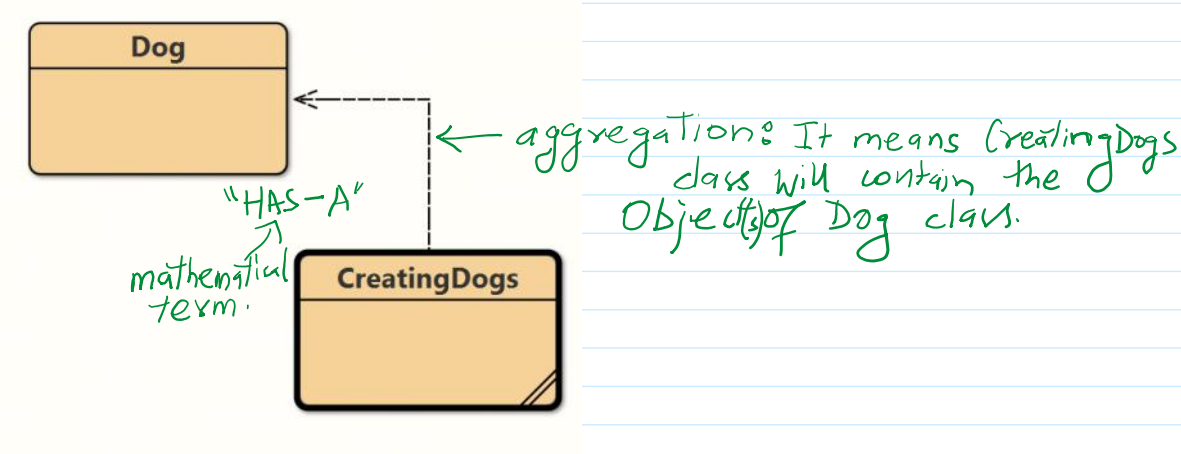


Syntax:

```
<class_name> variable = new <classname>();
```

new keyword in Java

The new keyword is used to allocate memory at runtime. All objects get memory in the heap memory area.
In Java, an instance of a class (also referred to as an object) is created using the new keyword. The new keyword dynamically allocates memory for an object of that class and returns a reference to it when it is followed by the class name and the bracket with optional arguments.

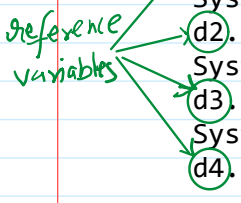


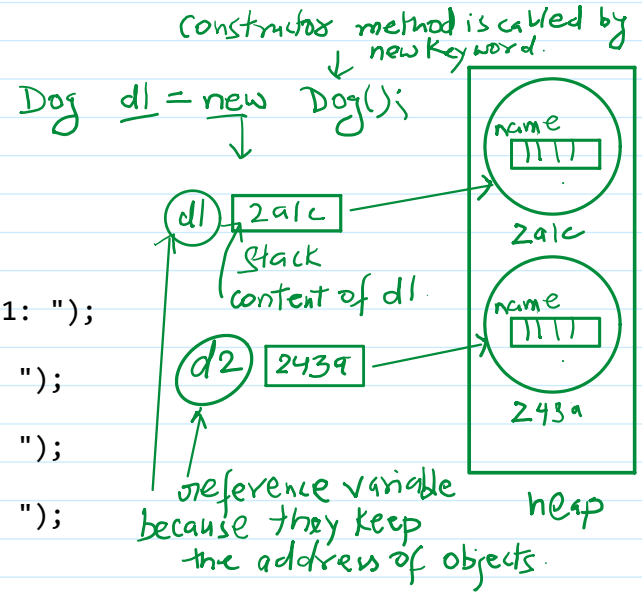

```
package OOP;
import java.util.*;


/**
 * Write a description of class CreatingDogs here.
 *
 * @author (your name)
```

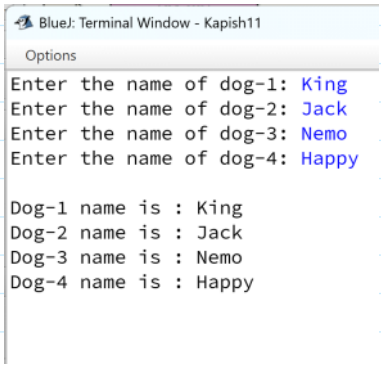
```
 * @version (a version number or a date)
 */
public class CreatingDogs
{
    public static void main(String[] args){
        Dog d1 = new Dog();
        Dog d2 = new Dog();
        Dog d3 = new Dog();
        Dog d4 = new Dog();
        Scanner sc = new Scanner(System.in);
        System.out.print("\fEnter the name of dog-1: ");
        d1.name = sc.nextLine();
        System.out.print("Enter the name of dog-2: ");
        d2.name = sc.nextLine();
        System.out.print("Enter the name of dog-3: ");
        d3.name = sc.nextLine();
        System.out.print("Enter the name of dog-4: ");
        d4.name = sc.nextLine();

        System.out.print("\nDog-1 name is : " + d1.name);
        System.out.print("\nDog-2 name is : " + d2.name);
        System.out.print("\nDog-3 name is : " + d3.name);
        System.out.print("\nDog-4 name is : " + d4.name);
    }
}
```





Output:



Initializing an Object in Java

There are the following three ways to initialize an object in Java:

1. By reference variable
2. By method
3. By Constructor

For example:

```
package OOP;


/**
 * Write a description of class Student here.
 *
 * @author (your name)
```

```
 * @version (a version number or a date)
 */
public class Student
{
    private int rollNumber;
    private String name;
    private byte std;

    public Student(){ //default constructor defined by the user
        rollNumber = 0;
        name = "";
        std = 0;
    }

    //parametrized constructor, used to initialize instance variable of
object
    public Student(int rn, String nm, byte std){
        this.rollNumber = rn;
        this.name = nm;
        this.std = std;
    }

    public void setRollNumber(int rollNumber){
        this.rollNumber = rollNumber;
    }

    public void setName(String nm){
        this.name = nm;
    }

    public void setStd(byte std){
        this.std = std;
    }

    public int getRollNumber(){
        return this.rollNumber;
    }

    public String getName(){
        return this.name;
    }

    public byte getStd(){
        return this.std;
    }
}

package OOP;
import java.util.*;


/**
 * Write a description of class CreateStudents here.
 *
 * @author (your name)
 * @version (a version number or a date)
```

```
 */
public class CreateStudents
{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.print("\f");
        Student s1 = new Student(); //by calling default constructor
        System.out.print("Enter roll number: ");
        s1.setRollNumber(sc.nextInt());
        sc.nextLine(); //to remove the keyboard buffer
        System.out.print("Enter name of student: ");
        s1.setName(sc.nextLine());
        System.out.print("Enter standard of student: ");
        s1.setStd(sc.nextByte());

        //Another object of student
        // using parameterized constructor to initialize it
        System.out.print("\nEnter roll number, name, and standard of
student: \n");
        int r = sc.nextInt();
        sc.nextLine();
        String nm = sc.nextLine();
        byte std = sc.nextByte();
        Student s2 = new Student(r, nm, std);

        System.out.print("\nStudent 1 info:");
        System.out.print("\nRoll number: " + s1.getRollNumber());
        System.out.print("\nName: " + s1.getName());
        System.out.print("\nStandard: " + s1.getStd());


        System.out.print("\nStudent 2 info:");
        System.out.print("\nRoll number: " + s2.getRollNumber());
        System.out.print("\nName: " + s2.getName());
        System.out.print("\nStandard: " + s2.getStd());
    }
}
```

the setXXXX() functions are used to initialize the instance variables of object

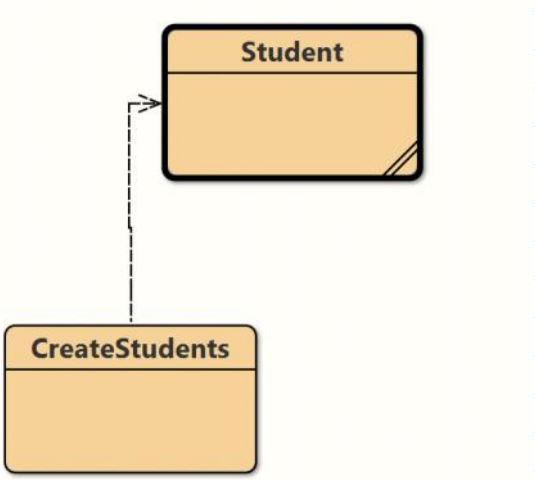
← parameterized constructor is used to initialize the object.

Output:

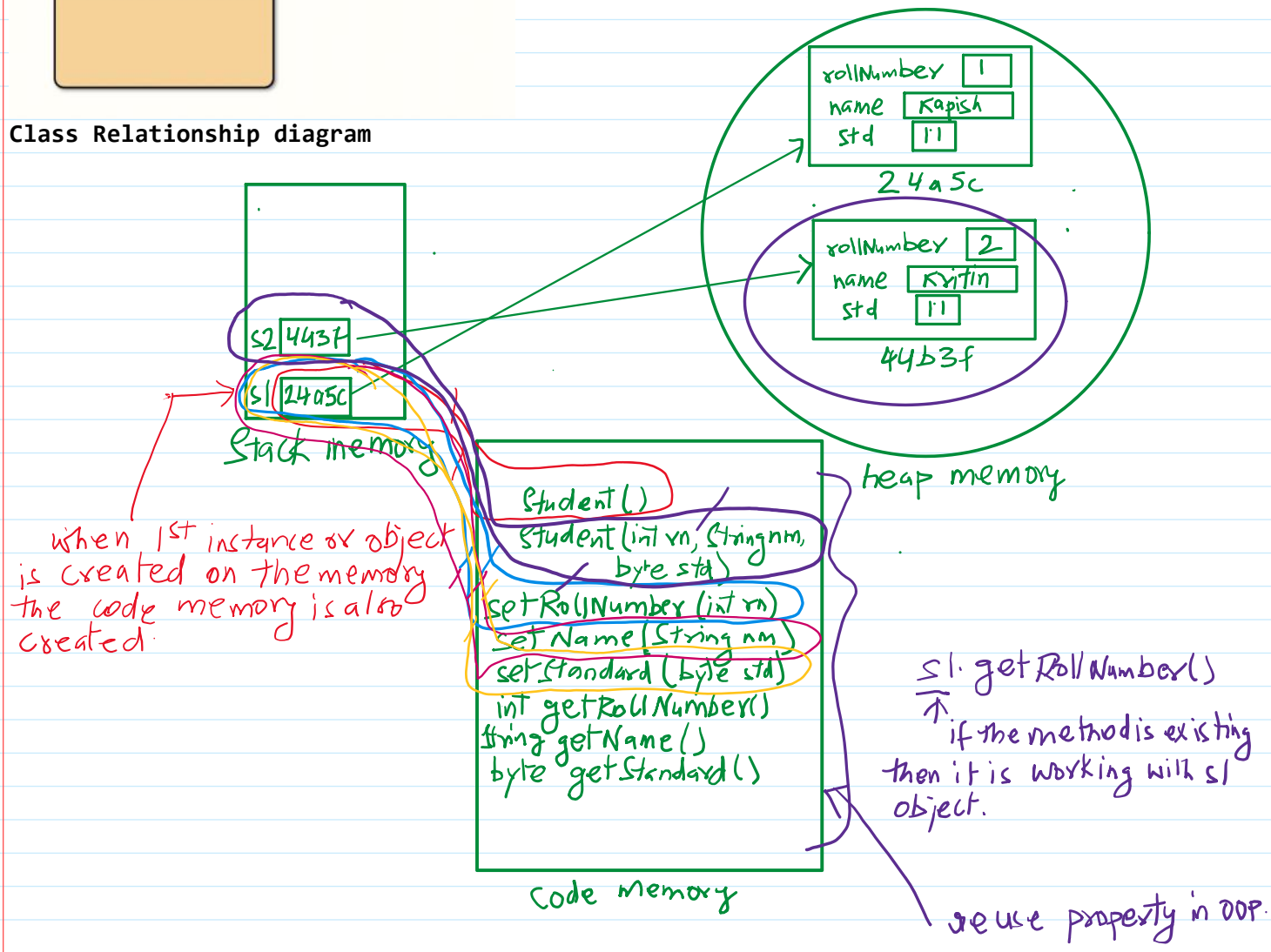
```
BlueJ: Terminal Window - Kapish11
Options
Enter roll number: 1
Enter name of student: Kapish
Enter standard of student: 11

Enter roll number, name, and standard of student:
2
Kritin
11

Student 1 info:
Roll number: 1
Name: Kapish
Standard: 11
Student 2 info:
Roll number: 2
Name: Kritin
Standard: 11
```



Class Relationship diagram



What are the different ways to create an object in Java?

1. **By new Keyword:** The most common way to create an object in Java is using the new keyword followed by a constructor. For example,

```
Scanner sc = new Scanner(System.in);
Student st = new Student();
```

The new keyword allocates memory for the object and calls its constructor to initialize it.

2. **By newinstance() Method(not in ISC):** This method is the part of the java.lang.Class class and used to create a new instance of a class

2. By newinstance() Method(not in ISC): This method is the part of the java.lang.Class class and used to create a new instance of a class dynamically at runtime. It invokes the number argument constructor of the class.

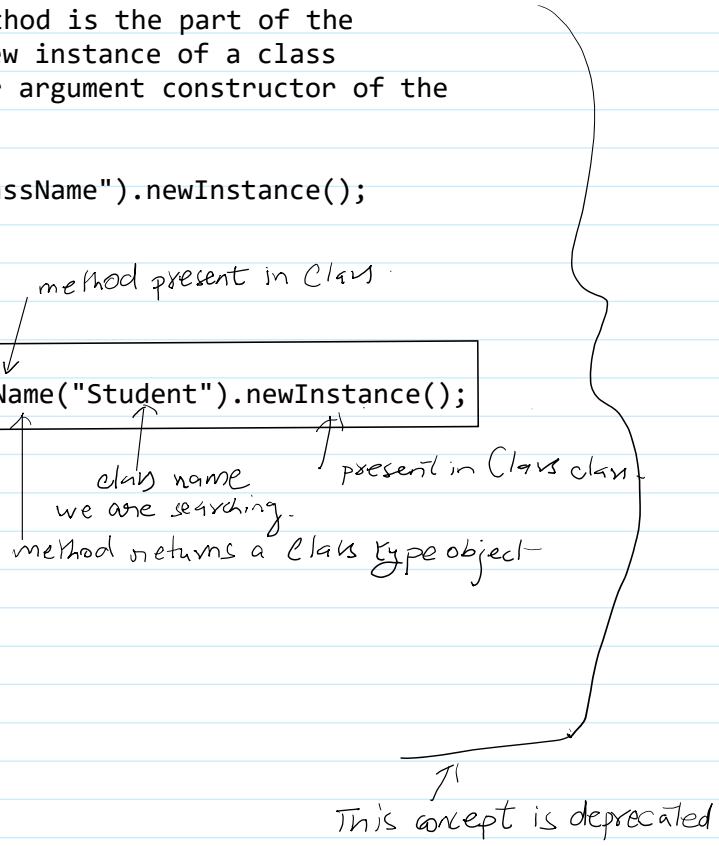
Syntax:

```
ClassName obj = (ClassName)Class.forName("ClassName").newInstance();
```

Example:

```
try
{
    try
    {
        Student s3 = (Student)Class.forName("Student").newInstance();
    }
    catch (InstantiationException ie)
    {
        ie.printStackTrace();
    }
}
catch (IllegalAccessException iae)
{
    iae.printStackTrace();
}
catch(ClassNotFoundException ob){

}
```



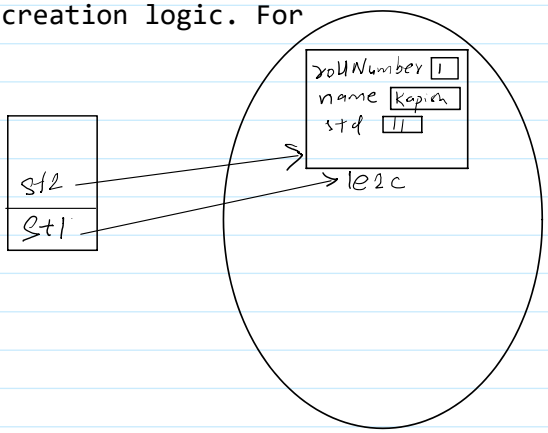
3. Factory method:

Factory methods are static methods within A class that return instances of the class. They provide a way to create objects without directly invoking a constructor and can be used to encapsulate object creation logic. For example:

```
//factory method
    public static Student newStudent(){
        return new Student();
    }

ClassName obj = ClassName.createInstance();

Student st6 = Student.newStudent();
```




```
    Student st1 = new Student(roll, nm, std);
    Student st2 = st1; //Create the reference of same student object
```